Assignment 1

Descriptive and Predictive Analytics for Imperials Ltd.

Module Code- MGT7219

Module Name- Data Mining

Submitted To - Dr. V. Charles

Student Name – Pradyuman Kumar

Student Number -40425764

Actual Word Count-2212 words excluding code and references

Introduction

“Predicting the future isn't magic, it's artificial intelligence.” These are words said by Dr Dave Waters, and in the times that we are living in the world AI is making the world move in almost all industries like automobile, aviation, healthcare insurance, etc.

The paper discusses the implementation of supervised learning in insurance. According to (Boodhun and Jayabalan, 2018) the technology that allows big data to be used for collection, processing, and analysis has revolutionized how the insurance sector uses big data to make good profits. as (Kurt *et al.*, 2022) the different ways that the sector uses supervised learning algorithms are in various sectors of insurance industries such as risk management, risk analysis, customer analytics, product development, marketing analytics, claims analytics, underwriting analytics, fraud detection, and reinsurance.

The paper talks about Imperials Ltd, a company in the insurance industry, that has a broad customer base and a varied array of products and services that are suited to their demands. Imperials Ltd has started a data-driven quest to increase sales of its life insurance product. It is using its vast customer database to build marketing techniques that are specifically targeted at its target audience. Within their customer base, Imperials Ltd seeks to discover and rank prospective life insurance purchasers using predictive modeling approaches. (Miller and Lim, 2020) Building predictive models that can identify trends in client behavior and anticipate their purchase inclinations is the main task of this assignment, which is a crucial step in reaching this goal. By combining data analytics with industry knowledge, the company hopes to obtain practical insights that will improve marketing efforts and encourage long-term consumer loyalty aiming to be at a position as an industry leader.

Methodology

The project's methodology entails several crucial processes to create and assess prediction models for locating possible customers for Imperials Ltd.'s life insurance product. To begin with, a Pandas Data Frame is loaded with the dataset that was taken from the client database of Imperials Ltd. for examination. To guarantee data quality and relevance, preprocessing operations such as feature selection, engineering, and cleaning are carried out (Brownlee, 2016). The train_test_split() function from scikit-learn is then used to divide the dataset into training and testing sets to make model evaluation easier (Hastie, Tibshirani and Friedman, 2009). In the model used in the paper, various classes are used to create three classification algorithms: logistic regression, decision tree classifier, and k-nearest neighbours (KNN). The

training data is used to train each model, and performance indicators including accuracy, ROC-AUC score, and Cohen's kappa score are used to assess each model. Interpreting model predictions and evaluating their effectiveness is made easier with the use of confusion matrices and visualizations produced using Matplotlib and Seaborn. Ultimately, the models are contrasted according to their assessment criteria to determine which model is most suited for forecasting life insurance purchases among the clientele of Imperials Ltd. References to pertinent literature and Python libraries facilitate the application and comprehension of the methodology. (Alexandre *et al.*, 2014)

Data Cleaning

It is an essential component of every data analysis effort, and this one is no exception. To prevent biased results, it is necessary to address the missing values and inconsistencies in the dataset.

This is the procedure we used to analyse the data:

1. **Child** – The data contained some value marked as 0 that has been replaced with ‘N’ that depicts the customer has ‘no child’

```
Y    18012
N    13333
U     8528
0      127
Name: child, dtype: int64
```

```
Y    18012
N    13460
U     8528
Name: child, dtype: int64
```

2. **Marriage**- Customer with their details having blanks in the status along with unknown marital status, has been updated keeping in mind the same people having kinds cannot be single or they are highly likely to be married. Customers with less than a high school diploma and ages less than or equal to 25 were more changed to singles.

```
dtype='object')
Married    20891
Single      5082
Name: marriage, dtype: int64
```

```
Married    20528
Single      4743
Name: marriage, dtype: int64
```

3. **Education –** The missing value from the row has been dropped as it might hinder the analysis missing value was 741.

```
2. Some College    11400
3. Bach             9267
1. HS               8828
4. Grad             5916
0. <HS              3848
Name: education, dtype: int64
Count of missing values in the 'education' column: 741
```

4. **Gender –** With the raw data we had values in the column that were labelled as U but as per the data dictionary it should not exist, so unkon values were removed keeping in mind that it might hinder the analysis.

```
M    22019
F    16830
U     1151
Name: gender, dtype: int64
```

```
M    22019
F    16830
Name: gender, dtype: int64
```

5. House Value - We found that there are some missing house values, which we can fill by finding the average house value for the area. Consequently, we filled in the missing house val values using the region. We separated the continuous variable 'home val' into 20 categories to help with management. The house value range was divided into twenty equal halves to create these groups.

Conversion of Data

Data Cleaning involves not just filtering the data but also preparing the data for further analysis. Which means converting the category variables to numerical variables. Most machine learning models need only numerical variables to run. Hence in the data cleaning variables. The "flag" variable was converted from having two categories—Y and N—to a binary variable, with Y equal to 1 and N equal to 0. The binary variable with M = 1 and F = 0 was created by converting the "gender" variable, which had two categories (M and F).

Five categories (0. HS, 1.HS, 2. Some College, 3.Bach, 4.Grad) made up the "education" variable. We converted it to a numerical variable with a number for each category (0, 1, 2, 3, 4). We started using education as a predictor in our machine-learning models.

Preparing data for analysis and machine learning algorithms involves converting category variables to numerical variables. It enables us to improve our models and forecast these variables.

```
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   flag         25753 non-null  int64
 1   gender       25753 non-null  int64
 2   education    25753 non-null  int64
 3   house_val    25753 non-null  int64
 4   age          25753 non-null  int64
 5   online       25753 non-null  int64
 6   marriage     25753 non-null  int64
 7   child        25753 non-null  int64
 8   occupation   25753 non-null  int64
 9   mortgage     25753 non-null  int64
 10  house_owner  23558 non-null  float64
 11  region       25753 non-null  int64
 12  fam_income   25753 non-null  int64
dtypes: float64(1), int64(12)
memory usage: 2.8 MB
None
```

```
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   flag         38849 non-null  object
 1   gender       38849 non-null  object
 2   education    38149 non-null  object
 3   house_val    38849 non-null  int64
 4   age          38849 non-null  object
 5   online       38849 non-null  object
 6   marriage     25474 non-null  object
 7   child        38849 non-null  object
 8   occupation   38849 non-null  object
 9   mortgage     38849 non-null  object
 10  house_owner  35610 non-null  object
 11  region       38849 non-null  object
 12  fam_income   38849 non-null  object
dtypes: int64(1), object(12)
memory usage: 4.1+ MB
None
```

Descriptive Statistics

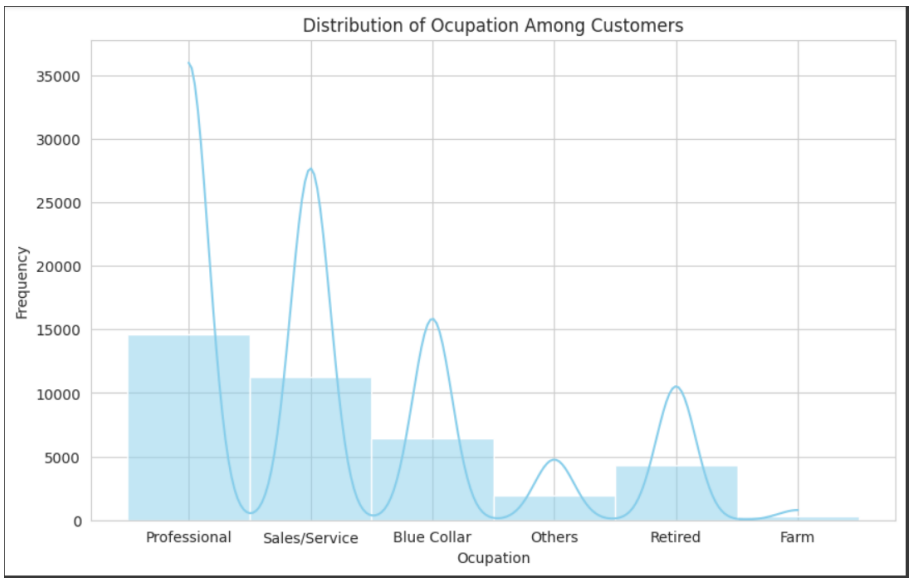
25271 observations total from 13 features in the dataset, including financial, geographic, and demographic information. A binary variable called "flag" with values of "Y" or "N" indicates whether the marketing campaign was receptive to the client. The majority of clients (51.59%) responded favourably to the marketing campaign, while 48.41% did not respond.

Further study of the data revealed that 75.87 percent of the consumers are homeowners. Most customers (57.47%) are married, and 66.15% of them have kids. Regarding employment, most clients (58.53%) are employed in white-collar jobs. Additionally, the dataset provides insightful data regarding the geographic, financial, and demographic characteristics of customers who either positively or negatively responded to the marketing campaign. These findings could be used to personalize future marketing campaigns to target client demographics and boost response rates.

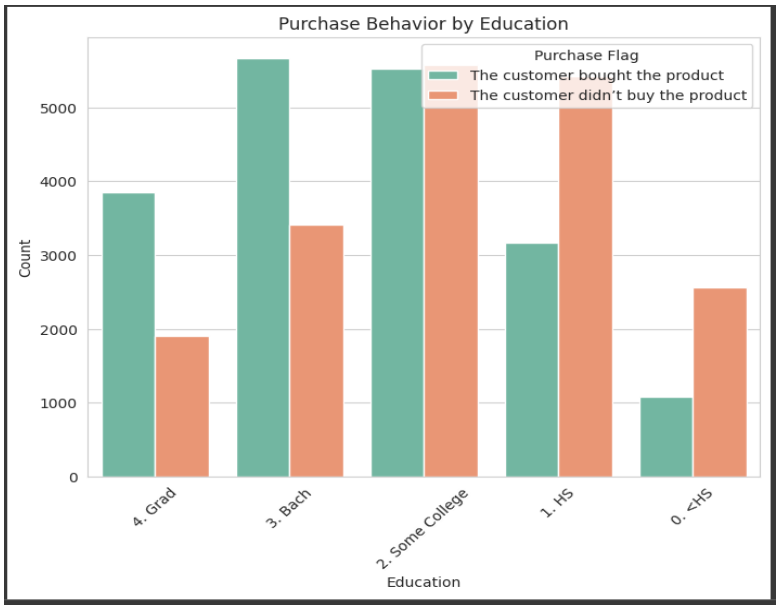
```
               flag        gender     education     house_val           age  \
count  25753.000000  25753.000000  25753.000000  2.575300e+04  25753.000000
mean       0.559042      0.637091      2.191085  3.173400e+05      4.266765
std        0.496511      0.518314      1.201969  4.349044e+05      1.782079
min        0.000000      0.000000      0.000000  0.000000e+00      1.000000
25%        0.000000      0.000000      1.000000  8.807200e+04      3.000000
50%        1.000000      1.000000      2.000000  2.243650e+05      5.000000
75%        1.000000      1.000000      3.000000  4.046840e+05      6.000000
max        1.000000      2.000000      4.000000  9.999999e+06      7.000000

             online      marriage         child    occupation      mortgage  \
count  25753.000000  25753.000000  25753.000000  25753.000000  25753.000000
mean       0.686134      0.805537      0.918767      4.089193      1.424067
std        0.464072      0.395794      0.721622      1.690251      0.736633
min        0.000000      0.000000      0.000000      1.000000      1.000000
25%        0.000000      1.000000      0.000000      4.000000      1.000000
50%        1.000000      1.000000      1.000000      4.000000      1.000000
75%        1.000000      1.000000      1.000000      6.000000      2.000000
max        1.000000      1.000000      2.000000      6.000000      3.000000

        house_owner        region    fam_income
count  23558.000000  25753.000000  25753.000000
mean       0.202776      3.240593      7.035491
std        0.402076      1.487826      2.728680
min        0.000000      1.000000      0.000000
25%        0.000000      2.000000      6.000000
50%        0.000000      4.000000      7.000000
75%        0.000000      4.000000      9.000000
max        1.000000      5.000000     12.000000
```

Visualization and Plotting

- **Plot 1: Distribution of Occupation among customers –** The Graph shows how the occupation x-axis, is distributed in the sales. The majority of the customers are from a professional background followed by a sales/service background, and the least customer procured was from a farm background and the people coming from another background, the representation aids in the understanding of the client's profession as well, it is very important because next time the Imperial Ltd knows that the main focus should be given to the Professional and the least to the people coming from other backgrounds and farms.

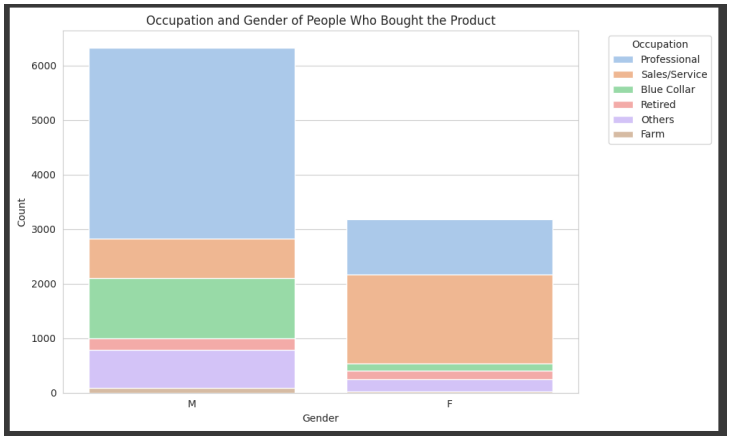


- **Plot 2: Purchase Behavior by Education-** The Graph shows the purchase pattern of people from different educational backgrounds, as shown that most people who are buying insurance from Imperials have just studied till their Bachelor's, on the other hand, the second most common category that of customer attended college but did not complete a full degree and the least Purchase has been done by Customer did not complete high school and that make sense as well because ideally students who did not complete high school are young and don't need insurance that that age, along with that the plot also depicts the customer do did not but it from the stores, and are also from the category of Customer did not complete high school.

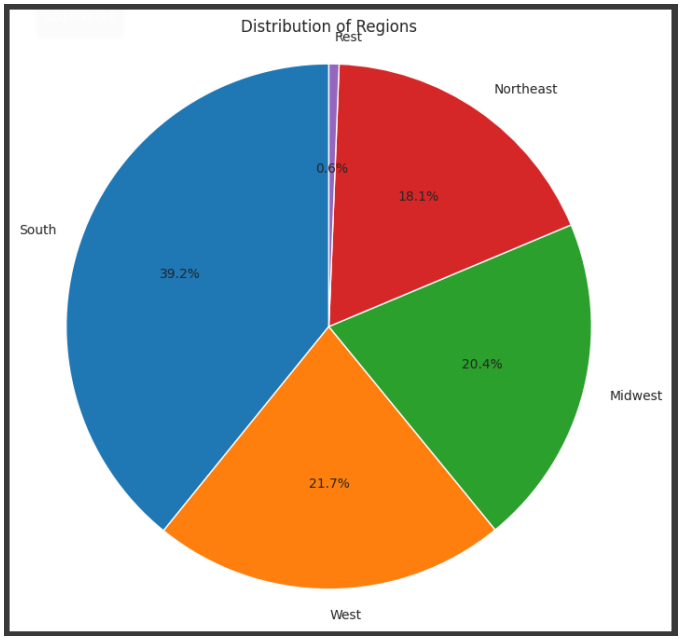


- **Plot 3: Purchase Behavior by Gender-** The plot depicts Gender-wise purchases in which we see that males are the ones that purchase the most the other Females buy the Products comparatively less. This insight could help the team to target largely men to increase sales and profit and will also help to understand the reason why the sale is comparatively less with females

- **Plot 4: Occupation and Gender of People Who Bought the Product-** The Bar chart depicts the occupation and Gender of People Who Bought the Product which means that one chart will show exactly whom to target in the next campaign to boost more Profitability and sales of the company, the plot shows that the people who buy the product are male and coming from professional background



- **Plot 5: Distribution of Regions –** The pie chart shows the distribution of regions from which major sales are occurring, in we get to see that the southern region with 39.2% customers is the highest consumer of the product and the west is the 2nd most insurance sold market with 21.7% and then 20.4% with Midwest, northeast with 18.1% and 0.6% with others. These graphs will help the company to target the south more to increase the market share in the south and the study why other regions are comparatively less.



Model Building:

- **Logistic Regression**

The dataset is split using a 70-30 split algorithm by the code into training and testing sets. After that, it uses the training data to train a logistic regression model and tests the model's performance using real data. Using a confusion matrix and a heatmap, the model's predictions are contrasted to the actual labels. To assess how well the model is working, accuracy is computed and reported as a percentage. Furthermore, to evaluate the model's discriminatory ability to differentiate between the classes, the area under the ROC curve (AUC) is calculated.

- **KNN classifier**

The dataset is divided into training and testing sets in this code. Using the fit() function, the KNeighborsClassifier model is trained on the training set (X_train, y_train). Next, using the predict() function, predictions are performed on the test data (X_test), and the model's performance is assessed by generating a confusion matrix. Accuracy_score() is used to compute the KNN Classifier's accuracy. Furthermore, roc_auc_score() is used to compute the Area Under the ROC Curve (AUC) score and predict probabilities for the test set. This makes it possible to assess the predictive ability and effectiveness of the model.

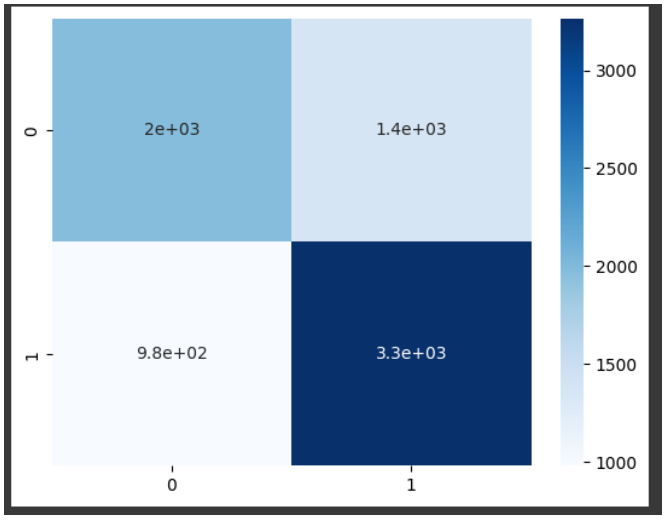
Model & Result

Model 1 & Result: Logistic Regression

The accuracy of the Logistic Regression model was 68.99% suggesting that it accurately classified approximately 7 out of 10 occurrences. The model AUC is 0.74. indicating that it has modest predictive power. The finding shows that the model may be useful for forecasting the outcome of interest. But furthermore, testing is required to establish its overall usefulness

In addition, the model predicted 1965 true negative instances (TN), 1372 true positive instances (TP), 979 false positive instances (FP), and 3266 false negative instances (FN) for the confusion matrix (FN).

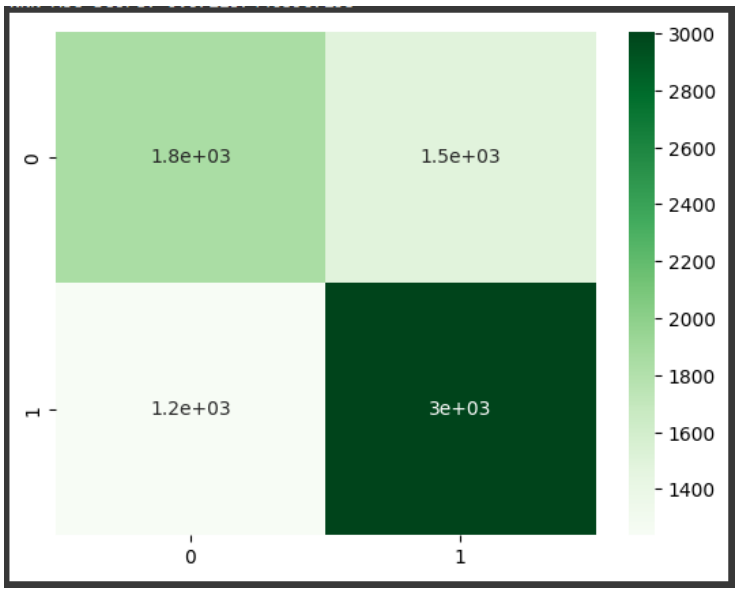
```
Confusion Matrix:
[[1965 1372]
 [ 979 3266]]
Logistic Regression Accuracy: 0.6899235030335004
Logistic Regression Accuracy: 68.99235030335004 %
LR AUC score: 0.7475799235681739
```



Model 2 & Result: KNN classifier

The model accurately classified 1965 occurrences as true negatives (TN), 1372 instances as true positives (TP), 979 instances as false negatives (FN), and 3266 instances as false positives (FP), according to the confusion matrix produced by the KNN Classifiers job. The model's total accuracy was found to be 64.05%, meaning that around two-thirds of the test set's cases could be predicted with accuracy in terms of their class labels. Furthermore, the KNN Classifiers model's AUC score of 0.67 shows that it has a respectable level of discrimination ability in differentiating between positive and negative classifications.

```
Confusion Matrix:
[[1965 1372]
 [ 979 3266]]
KNN Classifier Accuracy: 0.6405961487734108
KNN Classifier Accuracy: 64.05961487734108 %
KNN AUC score: 0.6721574465967295
```



Evaluating the Best Model

```
                  Model  Accuracy  F1-Score
0  Logistic Regression  0.689924  0.735337
1        Random Forest  0.652598  0.698834
2                  KNN  0.640596  0.688179
Best performing model: Logistic Regression
```

The best-performing model among the tested algorithms **is Logistic Regression**, with an accuracy of 68.99% and an F1-score of 73.54%. Logistic Regression outperforms KNN in both metrics, making it the preferred choice for predicting the target variable. Its simplicity and effectiveness in handling linear relationships make it the optimal model for this dataset**.**

Conclusion

In conclusion, Imperial Ltd's initiative is a very noble initiative to enhance life insurance sales through data-driven strategies that will be successful as the model made works with good accuracy. carrying out a predictive analysis on a dataset including details about the financial and demographic habits of the clients.

It was discovered that more than 51% of the customers had responded favorably to a previous

direct marketing initiative. There were thirteen variables in the dataset, representing both numerical and category data.

Effective identification of potential buyers was the goal of Imperials Ltd. The business obtained important insights into consumer behaviour and purchase patterns by creating and assessing machine learning models, such as logistic regression and KNN Classifiers.
It was shown that the logistic regression model performed better than the KNN Classifiers model in terms Rwith an accuracy of 68.99% and an F1-score of 73.54%. Accordingly, logistic regression appears to be the best model for forecasting consumer behaviour and locating possible customers.

Recommendation

Based on the analysis, The report suggests that Imperials Ltd. should focus on implementing logistic regression-based predictive models to effectively guide marketing campaigns and sales strategies. To ensure accurate forecasts and increase model performance, it is recommended to conduct ongoing monitoring and model improvement.

Along with that they also should work on the Research and survey of why Females, people from other professions, age and gender-specific are not buying the insurance from them so that they could grow more in number and profits. All things considered, Imperials Ltd. may be able to continue growing and succeeding in its life insurance industry by implementing data-driven strategies and cutting-edge analytics techniques.

References

Boodhun, N. and Jayabalan, M. (2018) 'Risk prediction in life insurance industry using supervised learning algorithms,' *Complex & Intelligent Systems*, 4(2), pp. 145–154. https://doi.org/10.1007/s40747-018-0072-1.

Kurt, Z. *et al.* (2022) 'Insurance sales forecast using machine learning algorithms,' in *Lecture notes in networks and systems*, pp. 29–38. https://doi.org/10.1007/978-981-19-0604-6_3.

Miller, J.D. and Lim, J. (2020) *Data-First Marketing: How To Compete and Win In the Age of Analytics*. John Wiley & Sons.

Cruz, M.G., Peters, G.W. and Shevchenko, P.V. (2015) *Fundamental aspects of operational risk and insurance analytics: A Handbook of Operational Risk*. John Wiley & Sons.

Hastie, T., Tibshirani, R. and Friedman, J. (2009) *The elements of statistical learning: Data Mining, Inference, and Prediction, Second Edition*. Springer Science & Business Media.

Brownlee, J. (2016) *Machine Learning Mastery with Python: Understand Your Data, Create Accurate Models, and Work Projects End-to-End*. Machine Learning Mastery.

Alexandre, A. *et al.* (2014) 'Machine learning for neuroimaging with scikit-learn,' *Frontiers in Neuroinformatics*, 8. https://doi.org/10.3389/fninf.2014.00014.

Kannan, K., & Subashini, R. (2019). A survey on data mining classifiers and its applications. In A Comparative Study of Classification Algorithms with Applications in Machine Learning (pp. 39-56). Springer. https://link.springer.com/chapter/10.1007/978-981-13-7961-3_3

Aljarah, I., Al-Jumaili, D., Al-Salem, A., Al-Qaysi, A., & Al-Hmouz, A. (2019). A survey of machine learning algorithms in big data. In Big Data and Internet of Things (pp. 31-57).

Springer. https://link.springer.com/chapter/10.1007/978-3-030-31713-4_2 Al-Kabi, A. M., & Saeed, A.M. (2019). A review of data mining in the health care domain: Challenges and future prospect. In Health Informatics and Big Data Analytics (pp. 31-54).

Springer. https://link.springer.com/chapter/10.1007/978-981-13-7911-6_3 D'Souza, A., & Kambhampati, C. (2005). The role of data mining in predicting customer churn in the telecommunications industry.

Communications of the Association for Information Systems, 16(1), 23-52. https://aisel.aisnet.org/cais/vol16/iss1/2/. Lopes, C., & Carvalho, J. (2019). Applying supervised machine learning algorithms to predict students' success in a Data Science course. In Proceedings of the 20th European Conference on Information Systems (pp. 1-10). https://aisel.aisnet.org/ecis2019sp/PresentingAuthor/123447/

Python Codes

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.metrics import cohen_kappa_score

from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier

from sklearn.ensemble import RandomForestClassifier


import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.linear_model import LogisticRegression

from sklearn.metrics import confusion_matrix, roc_auc_score, accuracy_score

import seaborn as sns

import matplotlib.pyplot as plt


from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import confusion_matrix, roc_auc_score, accuracy_score

import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.metrics import classification_report


from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.metrics import confusion_matrix, roc_auc_score, accuracy_score

import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import roc_auc_score
```

```
##Uploading Files

from google.colab import files

uploaded = files.upload()

##Storing and creating a data frame

My_data=pd.read_csv('sales_data.csv')

data1= pd.read_csv('sales_data.csv')
##Display data

total_rows_left = len(My_data)



print(f"\nTotal rows left in the DataFrame: {total_rows_left}")




print("\nDescriptive Statistics for categorical (object type) columns:")

print(My_data.describe(include=[object]))

# Count the missing values in the "education" column

missing_education_count = My_data['education'].isnull().sum()



print("Count of missing values in the 'education' column:", missing_education_count)

#Cleaning Data

My_data = My_data[My_data['age'] != 'U'] #Removing u from the agea
```

```
My_data = My_data.dropna(subset=['education']) # Removing the Blank Value from
        Education
My_data = My_data.dropna(subset=['marriage']) #Removing the blank value from Mrriage
My_data['child'] = My_data['child'].str.replace('0', 'N') #Replacing all the '0' with N
My_data = My_data[My_data['gender'] != 'U']

#Replace Blanks Places in marriage by single where value of Age is "2_<=25" and Education
        has value "0.<HS"

My_data['marriage'] = np.where((My_data['age'] == '2_<=25') &

(My_data['education'] == '0. <HS') &

(My_data['marriage'].isna()), 'Single', My_data.marriage)

#Replace all the Blanks in the Marriage with Marriage where the perosn has a chid.

My_data['marriage'] = np.where((My_data['age'].isin(['1_Unk','2_<=25','3_<=35', '4_<=45',
        '5_<=55',

'6_<=65', '7_>65'])) &

(My_data['child'] == 'Y') &

(My_data['marriage'].isna()), 'Married', My_data['marriage'])


#Finding mean value of the same region for replacing 0 in with the Estimated House Value

west_mean = round(My_data.house_val[My_data.region == "West"].mean())

west_mean

south_mean = round(My_data.house_val[My_data.region == "South"].mean())

south_mean

midwest_mean = round(My_data.house_val[My_data.region == "Midwest"].mean())
```

```
midwest_mean

northeast_mean = round(My_data.house_val[My_data.region == "Northeast"].mean())

northeast_mean

rest_mean = round(My_data.house_val[My_data.region == "Rest"].mean())

rest_mean

My_data.loc[(My_data['house_val'] == 0) & (My_data['region'] == 'West'), 'house_val']  =
        west_mean

My_data.loc[(My_data['house_val'] == 0) & (My_data['region'] == 'South'), 'house_val']  =
        south_mean

My_data.loc[(My_data['house_val'] == 0) & (My_data['region'] == 'Midwest'), 'house_val']
        = midwest_mean

My_data.loc[(My_data['house_val'] == 0) & (My_data['region'] == 'Northeast'), 'house_val']
        = northeast_mean

My_data.loc[(My_data['house_val'] == 0) & (My_data['region'] == 'Rest'), 'house_val']  =
        rest_mean

# Converting all Categorical Variable into Numerical Variable

My_data.gender = My_data.gender.map(dict(M=1, F=0, U=2))

My_data.flag = My_data.flag.map(dict(Y=1, N=0))

My_data.online = My_data.online.map(dict(Y=1, N=0))

My_data.marriage = My_data.marriage.map(dict(Married=1, Single=0))

My_data.child = My_data.child.map(dict(N=0,Y=1,U=2))

My_data.house_owner = My_data.house_owner.map(dict(Renter=1, Owner=0))
```

```
My_data.fam_income =
        My_data.fam_income.map(dict(A=12,B=11,C=10,D=9,E=8,F=7,G=6,H=5,I=4,J=3,K
        =2,L=1,U=0))

edu_map = {'0. <HS': 0, '1. HS': 1, '2. Some College': 2, '3. Bach': 3, '4. Grad': 4}

My_data['education'] = My_data['education'].map(edu_map)

age_mapping = {'1_Unk': 1, '2_<=25': 2, '3_<=35': 3, '4_<=45': 4, '5_<=55': 5, '6_<=65': 6,

'7_>65':7}

My_data['age'] = My_data['age'].map(age_mapping)

occupation_map = {'Blue Collar': 1, 'Sales/Service': 6, 'Farm': 2, 'Professional': 4, 'Others': 3,

'Retired': 5}

My_data['occupation'] = My_data['occupation'].map(occupation_map)

mortgage_map = {'1Low': 1, '2Med': 2, '3High': 3}

My_data['mortgage'] = My_data['mortgage'].map(mortgage_map)

region_map = {'Midwest': 1, 'South': 4, 'Rest': 3, 'West': 5, 'Northeast': 2,}

My_data['region'] = My_data['region'].map(region_map)


# Display column names, count of non-null values, and data types

print(data1.info())

# Generate descriptive statistics

descriptive_stats = My_data.describe()
```

```
# Display descriptive statistics

print(descriptive_stats)

#CREATING DUMMY OF EVERY VARIABLE

My_data = pd.get_dummies(My_data, columns=['gender'], prefix='gender',drop_first=False)

My_data = My_data.iloc[:, :-1]

My_data = pd.get_dummies(My_data, columns=['education'],
        prefix='education',drop_first=True)

My_data = pd.get_dummies(My_data, columns=['age'], prefix='age',drop_first=True)

My_data = pd.get_dummies(My_data, columns=['online'], prefix='online',drop_first=True)

My_data = pd.get_dummies(My_data, columns=['marriage'],
        prefix='marriage',drop_first=True)

My_data = pd.get_dummies(My_data, columns=['child'], prefix='child',drop_first=True)

My_data = pd.get_dummies(My_data, columns=['occupation'],
        prefix='occupation',drop_first=True)

My_data = pd.get_dummies(My_data, columns=['mortgage'],
        prefix='mortgage',drop_first=True)

My_data = pd.get_dummies(My_data, columns=['house_owner'],
        prefix='house_owner',drop_first=True)

My_data = pd.get_dummies(My_data, columns=['region'], prefix='region',drop_first=True)

My_data = pd.get_dummies(My_data, columns=['fam_income'],
        prefix='fam_income',drop_first=True)

My_data = pd.get_dummies(My_data,

prefix='house_val_category',drop_first=True)
```

```
#SCALE THE VARIABLE

scaler = StandardScaler()

scaled_data = scaler.fit_transform(My_data)

print(len(My_data.index))

print(len(My_data.values))

print(My_data.isnull().sum())

My_data.info()

My_data.to_csv('modified_dataset2.csv', index=False)

new_My_data= pd.read_csv('modified_dataset2.csv')

new_My_data.info()

new_My_data.flag.value_counts()

##########################MODEL 1: LOGISTIC REGRESSION

X = new_My_data.drop(['flag','house_val'], axis=1)

y = new_My_data['flag']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

C = 10

model = LogisticRegression()

model.fit(X_train, y_train)

y_pred_lr = model.predict(X_test)

# generate confusion matrix
```

```
cm = confusion_matrix(y_test, y_pred_lr)

print('Confusion Matrix:')

print(cm)

# plot the confusion matrix using heatmap

sns.heatmap(cm, annot=True, cmap='Blues')

accuracy = accuracy_score(y_test, y_pred_lr)

print("Logistic Regression Accuracy:", accuracy)

print("Logistic Regression Accuracy:", accuracy*100, "%")

# predict probabilities for the test set using the logistic regression model

y_pred_prob = model.predict_proba(X_test)[:, 1]

# calculate AUC score

auc = roc_auc_score(y_test, y_pred_prob)

print("LR AUC score:", auc)
#####################MODEL 2: KNN classifie

knn = KNeighborsClassifier(n_neighbors=5)

# fit the model with the training data

knn.fit(X_train, y_train)

# predict the class labels for the test data

y_pred_knn = knn.predict(X_test)

cm3 = confusion_matrix(y_test, y_pred_knn)

sns.heatmap(cm3, annot=True, cmap='Greens')

# Print the confusion matrix

print('Confusion Matrix:')
```

```
print(cm)
# calculate the accuracy of the model
accuracy = accuracy_score(y_test, y_pred_knn)
print("KNN Classifier Accuracy:", accuracy)
print("KNN Classifier Accuracy:", accuracy*100, "%")
# predict probabilities for the test set using the KNN model
y_pred_prob = knn.predict_proba(X_test)[:, 1]
# calculate AUC score
auc = roc_auc_score(y_test, y_pred_prob)
print("KNN AUC score:", auc)
##################EVALUATING THE BEST MODEL##########################
models = ['Logistic Regression', 'Random Forest', 'KNN']
accuracies = [accuracy_score(y_test, y_pred_lr), accuracy_score(y_test, y_pred_rf),
accuracy_score(y_test, y_pred_knn)]
f1_scores = [classification_report(y_test, y_pred_lr, output_dict=True)['1']['f1-score'],
classification_report(y_test,
y_pred_rf,
output_dict=True)['1']['f1-score'],
classification_report(y_test, y_pred_knn, output_dict=True)['1']['f1-score']]
results = pd.DataFrame({'Model': models, 'Accuracy': accuracies, 'F1-Score': f1_scores})
print(results)
# Find the best performing model based on the F1-score
best_model = results.loc[results['F1-Score'].idxmax(), 'Model']
print("Best performing model:", best_model)
#################Visualization 1
# Set style for Seaborn
```

```
sns.set_style("whitegrid")

# Define the bins for grouping Ocupation

# Create histogram plot

plt.figure(figsize=(10, 6))

sns.histplot(data=data1, x='occupation', bins=10, kde=True, color='skyblue')

plt.title('Distribution of Ocupation Among Customers')

plt.xlabel('Ocupation')

plt.ylabel('Frequency')

plt.grid(True)

plt.show()

#################Visualization 2

# Set style for Seaborn

sns.set_style("whitegrid")


# Create the countplot

plt.figure(figsize=(8, 6))

ax = sns.countplot(data=data1, x='education', hue='flag', palette='Set2')

plt.title('Purchase Behavior by Education')

plt.xlabel('Education')

plt.ylabel('Count')


# Custom legend labels

handles, labels = ax.get_legend_handles_labels()

ax.legend(handles=handles, labels=['The customer bought the product', 'The customer didn’t
buy the product'], title='Purchase Flag')
```

```
plt.xticks(rotation=45)  # Rotate x-axis labels for better readability
plt.show()
#################Visualization 3
import seaborn as sns
import matplotlib.pyplot as plt

# Set style for Seaborn
sns.set_style("whitegrid")

# Create the countplot
plt.figure(figsize=(8, 6))
ax = sns.countplot(data=data1, x='gender', hue='flag', palette='Set2')
plt.title('Purchase Behavior by Gender')
plt.xlabel('Gender')
plt.ylabel('Count')

# Custom legend labels
handles, labels = ax.get_legend_handles_labels()
ax.legend(handles=handles, labels=['Male', 'Female'], title='Gender')

plt.show()
import seaborn as sns
import matplotlib.pyplot as plt

# Filter the DataFrame to include only rows where the product was bought
bought_product = data1[data1['flag'] == 'Y']
```

```
# Set the style for seaborn
sns.set_style("whitegrid")

# Create a countplot with stacked bars
plt.figure(figsize=(10, 6))
sns.countplot(data=bought_product, x='gender', hue='occupation', palette='pastel',
dodge=False)
plt.title('Occupation and Gender of People Who Bought the Product')
plt.xlabel('Gender')
plt.ylabel('Count')
plt.legend(title='Occupation', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
import matplotlib.pyplot as plt

# Calculate the percentage of each region
region_percentage = data1['region'].value_counts(normalize=True) * 100

# Plotting the pie chart
plt.figure(figsize=(8, 8))
plt.pie(region_percentage, labels=region_percentage.index, autopct='%1.1f%%',
startangle=90)
plt.title('Distribution of Regions')
plt.axis('equal')  # Equal aspect ratio ensures the pie chart is circular
plt.show()
num_observations = len(My_data)
```

```
print("Number of observations:", num_observations)

# Assuming 'df' is your DataFrame

num_variables = My_data.shape[1]

print("Number of variables:", num_variables)
```